

F: Fingerprints

Source file name: `fingerprints.c`, `fingerprints.cpp`, `fingerprints.java`, or `fingerprints.py`

Author: Camilo Rocha

A forensic database stores fingerprints encoded as strings of characters. These encodings are stored *circularly* so that the starting position in the string is arbitrary. For example, the encodings ABCD, BCDA, CDAB, and DABC all represent the same fingerprint. However, DABC and ABDC represent different fingerprints.

Lately, the database is running out of cloud storage space---apparently most of it is being consumed by the ever-growing needs of modern GPTs. To clean up the storage, the forensic team wants to keep only one copy of each distinct fingerprint. To do so, each group of equivalent encodings will be represented by its *canonical encoding*: the lexicographically smallest rotation.

Given a list of fingerprint encodings, output the set of distinct fingerprints that remain after deduplication, each represented by its canonical encoding.

Input

The input consists of several test cases, each being a line containing a number N ($1 \leq N \leq 100$) of fingerprints. Then, N lines follow, each containing a blank-separated pair M, F , where M is the length of the fingerprint F ($M = |F|$ and $1 \leq M \leq 5\,000$). Each fingerprint consists of uppercase English characters only. The end of input is signaled with a line containing a single 0.

The input must be read from standard input.

Output

For each test case, output the canonical encodings of all distinct fingerprints, one per line, in the order in which they can be found in the input.

The output must be written to standard output.

Sample Input	Sample Output
5	ABCD
4 ABCD	XYZ
4 BCDA	FFFF
3 XYZ	
4 FFFF	
3 ZXY	
0	